CMSC433 / Part 04

Monad tracing

Topic mock

Topic mock, using the sample's question styles. Attempt independently; feedback after the whole file. Untimed practice, no points assigned. Current exam scope and rules are unconfirmed.

Question 1. Evaluate each expression using the program below. Leave function values such as `id`, `not`, and `const True` unevaluated. All parts are independent; preserve list enumeration order.

```
pack :: Monad m => [m (a -> b)] -> m [a -> b]
pack [] = return []
pack (u:us) = do
  f <- u
  fs <- pack us
  return (f:fs)
```

(a) `pack [Just not]`

Answer: [Result.]

(b) `pack [Just not, Nothing]`

Answer: [Result.]

(c) `pack [[] ::  [Bool -> Bool]]`

Answer: [Result.]

(d) `pack [[id,not], [const True]]`

Answer: [Result in order.]

Question 2. Rewrite `pack` using exactly one call to `mapM`, with no explicit recursion. Write your Haskell expression below.

```
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
```

```
packViaMapM :: Monad m => [m (a -> b)] -> m [a -> b]
packViaMapM = undefined  -- YOUR ANSWER
```